ФСП Карьера — сопроводительная документация

Платформа подбора ИТ-специалистов с обратной механикой: кандидат подтверждает уровень тестом и получает категорию, а работодатель сам находит нужную категорию и выходит на конкретного человека с предложением и зарплатной вилкой.

Документ описывает архитектуру решения, механики тестирования и подбора, процедуру валидации и её результаты, схему интеграции с реестром ФСП, API, используемые компоненты и порядок запуска.

1. Ключевая идея

На обычных площадках кандидат откликается вслепую, а работодатель вручную отсеивает нерелевантные отклики. Причина в том, что резюме описывает человека с его слов и не подтверждает уровень. Платформа меняет обе части:

1. **Категорию определяет тест, а не резюме.** Кандидат проходит опрос (специализация, стек, предполагаемый грейд) и адаптивный тест. Категория — связка «специализация × грейд» — присваивается только по результату теста.
2. **Инициатива у работодателя.** Он описывает потребность, получает подборку с объяснением и отправляет приглашение конкретному кандидату. В приглашении обязательны описание предложения, зарплатная вилка, название компании и способ связи — кандидат видит условия до начала общения.
3. **Контакты кандидата закрыты.** Работодатель видит обезличенную карточку: категорию, балл теста, стек, достижения ФСП. Имя и контакты открываются, только когда кандидат принял приглашение или откликнулся сам.

Сквозной сценарий:

```
Кандидат:     регистрация или вход через ФСП ID → опрос → адаптивный тест → категория → публикация профиля
Работодатель: описание потребности → подборка с объяснением → приглашение (вилка, условия)
Кандидат:     просмотр приглашения → принять / отклонить
Работодатель: видит статус; после принятия — имя и контакты кандидата
```

2. Функциональная архитектура

2.1. Кабинет кандидата

Раздел	Что делает
Тест и категория	Опрос, адаптивный тест, текущая категория, история попыток, правила повторного прохождения
Приглашения	Входящие предложения с компанией, вилкой и способом связи; принять или отклонить
Вакансии	Открытые вакансии, самостоятельный отклик, статусы своих откликов
Задания	Короткие задачи от работодателей по специализации кандидата, свои ответы и оценки
Профиль	Ручное заполнение либо распознавание резюме из PDF; выгрузка стандартизированного PDF-профиля
Настройки	Публикация профиля, что видно работодателю, привязка ФСП ID через вход в него, отзыв согласия с удалением данных

2.2. Кабинет работодателя

Раздел	Что делает
Подборки	Описание потребности текстом, разбор в структуру, рекомендованные категории и ранжированные кандидаты с объяснением, уточняющие фильтры
Банк кандидатов	Вся база с фильтрами по специализации, грейду, стеку, формату и наличию достижений ФСП; кандидаты с неподтверждённым грейдом — отдельным списком в конце

Раздел	Что делает
Приглашения	Отправленные приглашения, их статусы, контакты принявших
Вакансии	Публикация и правка вакансий, входящие отклики, принять или отклонить
Задания	Публикация коротких задач, обезличенные ответы кандидатов, оценка ответов
Компания	Описание организации и контактные данные

2.3. Правила видимости

Сведения о кандидате	Когда видны работодателю
Категория, балл и дата теста, стек	Всегда, если профиль опубликован
Заявленный грейд, балл и дата непройденного теста	Если профиль опубликован, а категории ещё нет (статус «грейд не подтверждён»)
Город, стаж, формат работы, «О себе», достижения и разряд ФСП	Если кандидат не скрыл их в настройках
Имя, почта, телефон, Telegram	После принятия приглашения либо собственного отклика кандидата
Ожидаемая зарплата, ответы на задания теста	Никогда

Кандидат видит только свои приглашения, отклики и ответы; чужая запись неотличима от несуществующей (ответ 404). Работодатель видит только свои потребности, вакансии, приглашения и задания.

Приглашения и отклики проходят статусы «отправлено → просмотрено → принято / отклонено». Статус «просмотрено» ставится, когда адресат открывает список; обе стороны видят актуальный статус.

3. Компонентная архитектура

```
Браузер (кандидат / работодатель)
        |  HTTPS
        v
Frontend: nginx + одностраничное приложение (React, TypeScript, Mantine)
        |  /api/*  (тот же источник, httpOnly-cookie)
        v
Backend: FastAPI, монолит с модулями
        |-- auth          регистрация, подтверждение почты, вход по паролю, отзыв согласия
        |-- reference     справочники: специализации, грейды, форматы, словарь технологий
        |-- candidates    профиль, разбор резюме из PDF, PDF-профиль, приватность
        |-- assessment    банк заданий, адаптивный тест (IRT), категории
        |-- matching      разбор потребности, ранжирование, подборки
        |-- invitations   приглашения и раскрытие контактов
        |-- vacancies     вакансии и отклики
        |-- challenges    регулярные задания
        |-- fsp           вход через ФСП ID и реестр достижений: контракты и их моки
        |-- employers     профиль компании
        v
PostgreSQL (в Docker)  /  SQLite (локальная разработка)

Внешние компоненты: SMTP-сервер (в Docker — почтовая ловушка Mailpit), ФСП ID и реестр ФСП (сейчас моки).
```

Монолит выбран осознанно: решение делает один разработчик, а модули связаны слабо и общаются через явные импорты моделей. Вычислительные ядра — `assessment/adaptive.py` (оценка уровня и выбор заданий) и `matching/ranking.py` (оценка соответствия) — не зависят ни от базы данных, ни от веб-слоя. Благодаря этому валидация прогоняет ровно тот же код, что работает в приложении, и при необходимости эти модули выносятся в отдельные сервисы без переписывания.

Интерфейс оформлен по дизайн-системе ФСП России, собранной из брендбука: цвета, шрифт JetBrains Mono, радиусы, логотип и фирменные линии-связи. Токены системы лежат в `frontend/src/styles/tokens.css`, тема компонентов — в `frontend/src/theme.ts`.

Точки расширения:

- `matching/parser.py: parse_need` и `candidates/resume.py: parse_resume` — разбор текста правилами; заменяются вызовом языковой модели с тем же форматом результата;
- `fsp/idp.py: FspIdProvider` — контракт провайдера входа ФСП ID; мок заменяется клиентом Keycloak;

- `fsp/registry.py: FspRegistry` — контракт реестра ФСП; мок заменяется клиентом реального API;
- `assessment/bank*.py` — банк заданий пополняется без изменения кода теста.

3.1. Модель данных

Таблица	Содержимое
users	Почта, хеш пароля, роль, подтверждение почты, дата согласия на обработку данных
auth_sessions	Открытые сессии входа: идентификатор из токена доступа, пользователь, срок действия
candidate_profiles	Самоописанный профиль, флаг публикации, скрытые поля, ФСП ID, копия достижений и разряд
companies	Профиль компании и контакты
assessments	Попытка теста: выданные задания с ответами, оценка уровня и её ошибка, итог
candidate_categories	Подтверждённая категория: специализация, грейд, балл, дата, предложение повышения
item_exposures	Счётчик показов каждого задания банка
item_flags	Задания, снятые с выдачи мониторингом дрейфа (раздел 4.4)
needs	Потребность работодателя: исходный текст и разобранная структура
invitations	Приглашение: предложение, вилка, способ связи, статус
vacancies, applications	Вакансии и отклики на них
challenges, challenge_answers	Регулярные задания и ответы с оценками

Схема создаётся при старте приложения. Типы колонок переносимы между PostgreSQL и SQLite.

4. Механика тестирования

4.1. Задача

Единый набор заданий быстро расходится между кандидатами. Генерация уникальных заданий нейросетью не гарантирует сопоставимой сложности. Нужен подход, при котором задания у кандидатов разные, а сложность и оценка сопоставимы.

4.2. Решение: параметризованный банк и адаптивный тест

Задание банка — генератор, а не готовый текст. За каждым заданием закреплены шаблон и уровень сложности, а числа, структура кода, имена переменных и набор вариантов ответа выбираются случайно при каждой выдаче. Сложность принадлежит шаблону и уровню, поэтому два варианта одного задания сопоставимы.

Состав банка:

Тип заданий	Количество	Что меняется от выдачи к выдаче
Общие: циклы, сложность алгоритмов, системы счисления, поиск, структуры данных, рекурсия, логика	28 (7 шаблонов × 4 уровня)	Числа, структура кода, имена, выражения — ответ пересчитывается
Профильные на термины	4 на специализацию, в каждом 8 терминов	Термин, направление вопроса (термин → определение и обратно), неверные варианты
Профильные ситуационные	8 на специализацию	Набор и порядок вариантов; текст вопроса фиксирован

Ответ на задания с кодом вычисляется выполнением того же кода, который показан кандидату, поэтому не может с ним разойтись.

Оценка — по модели IRT (3PL). Вероятность верного ответа зависит от уровня кандидата θ и параметров задания: сложности b, различающей способности a и вероятности угадать c = 0,25 (четыре варианта ответа). Шкала θ привязана к грейдам: нижняя граница стажёра −2, junior −1, middle 0, senior +1. Уровень оценивается как среднее апостериорного распределения (EAP), балл для показа: 50 + 15·θ, то

есть границы грейдов — 20 / 35 / 50 / 65.

Тест адаптивный и переменной длины. Тест отвечает на один вопрос: уровень кандидата не ниже границы заявленного грейда или ниже. Задания выдаются по одному, каждое третье — профильное; следующее выбирается среди самых информативных в точке этой границы, где решение и принимается. После 12 заданий тест заканчивается, как только апостериорная вероятность «уровень не ниже границы» достигает 90% или падает до 10%; кандидат у самой границы получает дополнительные задания, но не больше 18. В среднем выходит 14,9 задания: 43% тестов заканчиваются на двенадцати, 41% доходят до восемнадцати. Вернуться к предыдущему заданию нельзя, на тест отводится 30 минут.

Политика выбрана по симуляции (раздел 6). По сравнению с прежним тестом из 15 заданий, подобранных под текущую оценку уровня, категория совпадает с истинным грейдом чаще (75% против 72%), а прошедших с уровнем ниже заявленного меньше (23% против 26%) при той же средней длине. Цена — балл теста стал грубее: задания у границы грейда хуже различают кандидатов далеко от неё, поэтому корреляция балла с истинным уровнем снизилась с 0,85 до 0,79, а надёжность — с 0,74 до 0,65. Балл участвует в ранжировании только как часть силы профиля, и на качестве подборки это не сказалось (раздел 6.2).

4.3. Правила грейда

- Тест пройден, если оценка уровня не ниже границы заявленного грейда.
- **Грейд не понижается принудительно.** Не прошедший тест может сразу сдать тест на грейд ниже; имеющаяся категория при провале сохраняется.
- **Повышение.** Если результат дотянул до следующего грейда, кандидат получает одну досрочную попытку на него.
- **Ограничение частоты.** Сменить подтверждённую категорию можно раз в 90 дней; после провала этот грейд и грейды выше закрыты на 14 дней — только в той специализации, по которой был тест: провал по тестированию не закрывает тот же грейд по бэкенду. Сроки задаются настройками `GRADE_CHANGE_COOLDOWN_DAYS` и `RETAKE_COOLDOWN_DAYS`.
- **Грейд не подтверждён.** Кандидат без категории, не прошедший тест на заявленный грейд, из выдачи не исчезает: он показывается работодателю со статусом «грейд не подтверждён» ниже подтверждённых (раздел 5.6).

Почему правила такие:

- **Грейд выбирает кандидат, тест только подтверждает.** Тест отвечает на вопрос «дотягивает ли до заявленного», а не раздаёт грейды сам: автоматическое повышение по одному удачному прохождению при надёжности теста 0,65 (раздел 6) давало бы случайные завышения. Поэтому результат выше грейда даёт не повышение, а право на отдельный тест следующего уровня.
- **90 дней на смену категории.** Категория — то, по чему работодатель ищет и на что опирается в приглашении; если она меняется каждые несколько дней, ей нельзя доверять. Квартал — срок, за который уровень специалиста действительно может измениться, и он совпадает с порогом актуальности профиля в ранжировании (раздел 5.3).
- **14 дней после провала.** Срок отсекает перебор попыток: при вероятности случайно пройти тест чужого грейда в несколько процентов (раздел 6) десяток попыток подряд дал бы незаслуженную категорию. Двух недель при этом хватает, чтобы подтянуть знания перед честной пересдачей.
- **Провал не сжигает попытку на грейд ниже и не отнимает имеющуюся категорию.** Ошибка в самооценке — штатная ситуация: кандидат сразу сдаёт тест уровнем ниже и не выпадает из подбора на две недели.

На стенде проверки включается демо-режим (`DEMO_MODE=true`, раздел 11.5): сроки действуют и видны в интерфейсе, но кандидат может сбросить свои попытки и категорию и пройти тест заново — так эксперт проходит тест несколько раз подряд и сравнивает результаты.

4.4. Почему механика устойчива к распространению заданий

1. **Заучивать нечего.** У общих заданий от 71 до нескольких сотен разных вариантов на 300 выдач, и верный ответ меняется вместе с вариантом. Пересказ «вопроса и ответа» другому кандидату бесполезен; чтобы ответить, нужно уметь решать такой тип задач.
2. **Заучить термины — значит выучить глоссарий.** В заданиях на термины каждый раз другой термин и другие неверные варианты.
3. **Тесты разные по составу.** Задание выбирается случайно из четырёх самых информативных, с перевесом в пользу реже показанных. Общее задание встречается примерно в 39% тестов.
4. **Повторы исключены.** Задания, которые кандидат видел в прошлых попытках, ему не выдаются, пока хватает новых.
5. **Утёкшие задания снимаются с выдачи.** Мониторинг дрейфа (`assessment/drift.py`) после каждого завершённого теста сверяет профильные задания с ожидаемой решаемостью. Уровень кандидата для этой сверки берётся только по общим заданиям попытки — их условия генерируются, и ответы на них не утекают. Если задание решают чаще ожидаемого больше чем на 15 п.п. (не меньше 30 ответов, превышение больше 2,5 стандартного отклонения), оно попадает в `item_flags` и больше не выдаётся; когда неснятых профильных заданий не хватает, их место занимают общие. Вернуть задание в банк можно только вручную, после того как его переписали.

Количественная проверка — в разделе 6: слабый кандидат, заучивший ответы из 10 чужих тестов, проходит в 32% случаев против 6% без утечки; при едином наборе заданий он проходил бы наверняка. При утечке 200 тестов мониторинг снимает 7 из 12 профильных заданий специализации, и доля проходящих списывающих падает с 83% до 46%; на честных ответах из 84 заданий напрасно не снято ни одного.

4.5. Ограничения

- Параметры сложности назначены экспертно и на реальных ответах не калибровались. Для калибровки предусмотрен режим `python -m scripts.validate_testing --db`, считающий по накопленным ответам долю верных, ожидаемую долю, индекс дискриминации и признак дрейфа каждого задания.
- Ситуационные профильные вопросы имеют фиксированный текст и остаются самым уязвимым к утечке местом. Мониторинг дрейфа снимает утёкшие задания, но срабатывает после накопления ответов и не отличает утечку от задания, которое просто легче, чем оценил эксперт; основная мера — расширение банка.
- Балл теста зависит от того, на какой грейд сдавался тест: задания подбираются у его границы, и оценка кандидата далеко от неё сжимается к ней. Сравнивать баллы корректно внутри одной категории — так они и используются в ранжировании.
- Точность одного теста ограничена его длиной и размером банка (см. раздел 6).

5. Механика подбора и категоризация

5.1. Категория

Категория — пара «специализация × грейд» из справочника: 7 специализаций (бэкенд, фронтенд, мобильная разработка, тестирование, DevOps, Data Science и ML, аналитика) и 4 грейда (стажёр, junior, middle, senior). Она присваивается только по пройденному тесту и хранится отдельно от самоописанного профиля. Достижения ФСП на грейд не влияют: они упорядочивают кандидатов внутри категории. В подборку попадают кандидаты с категорией, разрешившие публикацию; кандидаты, чей грейд не подтверждён, показываются после неё отдельным списком (раздел 5.6).

5.2. Потребность работодателя

Работодатель описывает потребность свободным текстом. Разбор выполняется правилами и словарями:

- **специализация** — по явным названиям и по типичным для неё технологиям;
- **грейд** — по названию либо по требуемому стажу;
- **стек** — по словарю из 75 технологий с вариантами написания («постгрес», «Postgres» → PostgreSQL);
- **формат работы** и **зарплатная вилка**.

Работодатель видит, что и по какому признаку распознано, и может поправить любое поле. Потребность сохраняется, а подборка пересчитывается при каждом открытии, поэтому новые кандидаты появляются в ней сами.

Языковая модель не используется: доступа к ней в рамках хакатона не было. Правила воспроизводимы и объяснимы, но хуже понимают нестандартные формулировки; замена разбора на модель предусмотрена архитектурой.

5.3. Ранжирование

В подборку входят кандидаты запрошенной специализации с запрошенным или смежным грейдом. Итоговая оценка от 0 до 100:

Часть	Вес	Составляющие
Соответствие потребности	85%	грейд 0,40 (смежный — половина); покрытие требуемого стека 0,45 (близкая технология — половина); формат работы 0,15
Сила подтверждённого профиля	до 15%	балл теста 0,60; достижения ФСП 0,25; актуальность профиля 0,15

- Незаданные требования не учитываются и оценку не занижают.
- Близкая технология закрывает требование наполовину: MySQL при запросе PostgreSQL, RabbitMQ вместо Kafka, Vue вместо React. Группы аналогов заданы в словаре технологий (`reference/skills.py`) и намеренно узкие. В объяснении такая замена показывается отдельной строкой, а не как совпадение.
- Сила профиля упорядочивает кандидатов с равным соответствием, но не перевешивает разницу в одном требовании — грейде, формате работы или одной технологии стека. Для этого её вес ограничен 90% «цены» самого мелкого требования запроса: когда заданы грейд, формат и стек из

трёх технологий, это 11,5%, из десяти — около 3,5%. Поэтому кандидат с полным покрытием стека всегда выше кандидата того же грейда, которому не хватает технологии. Разницу между требуемой и близкой технологией (половина требования) сила профиля перевесить может. Соотношение между разными требованиями (например, грейд против нескольких технологий) задают веса соответствия, а не сила профиля. Верхняя граница 15% выбрана по результатам валидации: при доле 40% в топ чаще попадали сильные, но хуже подходящие под запрос кандидаты.
- Достижения ФСП дают надбавку по числу соревнований, результату и давности: победа — 0,4, призовое место — 0,3, результат старше двух лет — с коэффициентом 0,6, сумма не больше 1. Уровень соревнования и дисциплина на вес не влияют — все соревнования равно важны. Участие без места весит 0,08, а все участия вместе — не больше 0,2: профиль нельзя «набить» одними явками, двадцать участий слабее одного призового места. Командный результат засчитывается участнику целиком: личный вклад реестр не выделяет. **Отсутствие истории ФСП — нейтральное состояние**: надбавки нет, штрафа нет, в объяснении это не упоминается.
- Актуальность — давность теста или последнего ответа на задание работодателя: полная в первые 90 дней, затем снижается.
- Банк кандидатов без потребности сортируется только по силе профиля.

5.4. Объяснимость

К каждому кандидату в выдаче прилагаются:

- итоговая оценка и её разбивка по составляющим;
- причины текстом, каждая со знаком: довод за кандидата, довод против или сведение, не меняющее оценку. Работодатель видит не только сильные стороны, но и то, чего кандидату не хватает. Примеры:
- за: «Категория совпадает с запросом», «Стек: 2 из 3 — Python, PostgreSQL», «Тест на грейд пройден, балл 58 из 100 — выше, чем у 64% кандидатов категории», «Достижения ФСП: соревнований — 3, призовых мест — 1»;
- против: «Нет в стеке: Kafka», «Формат работы не совпадает с пожеланиями кандидата», «Уровень подтверждён давно: последняя активность 210 дн. назад»;
- сведение: «Близкий опыт: RabbitMQ вместо Kafka».

Место среди кандидатов категории показывается, когда в ней не меньше пяти человек.

5.5. Уточнение без потери подборки

Поверх подборки накладываются фильтры: грейд, обязательный стек, формат работы, только с достижениями ФСП. Они не меняют ни потребность, ни оценки, ни порядок; сброс возвращает исходную подборку. Рекомендованные категории показываются с числом доступных кандидатов и работают как быстрый фильтр.

5.6. Кандидаты с неподтверждённым грейдом

Кандидат, который заявил грейд, но тест на него не прошёл и категории не имеет, остаётся в выдаче: иначе до пересдачи он полностью выпадал бы из оборота. При этом заявленный грейд ничем не подкреплён, поэтому смешивать таких кандидатов с подтверждёнными нельзя.

- Они показываются отдельным списком **после** подборки с пометкой «грейд не подтверждён». Итоговая оценка им не выставляется, поэтому никакое совпадение стека не поднимает их выше подтверждённого кандидата.
- Отбор тот же, что у подтверждённых: та же специализация, заявленный грейд — запрошенный или смежный.
- В карточке — статус с фактами: какой грейд заявлен, когда не пройден тест, балл попытки и порог грейда («балл 41 из 100 при пороге 50»), а также покрытие стека и достижения ФСП. Порядок внутри списка — по покрытию стека, затем по баллу попытки.
- Число кандидатов в рекомендованных категориях и размер подборки считаются только по подтверждённым.
- Пригласить такого кандидата можно на общих основаниях; контакты так же закрыты до принятия приглашения.
- Кандидат, который тест ещё не сдавал, грейда не заявлял и в выдачу не попадает. Кандидат с категорией, не прошедший тест на грейд выше, показывается со своей подтверждённой категорией.

6. Валидация

6.1. Процедура

Эталонной разметки нет, поэтому она задаётся генератором синтетических кандидатов (`backend/scripts/synth.py`). У каждого кандидата есть скрытая истина — уровень θ и реально освоенные технологии — и наблюдаемые данные: заявленный грейд (60% честны, 35% завышают, 5% занижают) и заявленный стек (с пропусками и «приписками»). Платформа видит только наблюдаемое и ответы на задания; качество измеряется сравнением её выводов со скрытой истиной.

Ответы на задания генерируются по модели IRT. Чтобы не проверять модель на самой себе, истинные

параметры заданий намеренно отличаются от назначенных: сложность сдвинута со стандартным отклонением 0,4 грейда, различающая способность — на ±25%. Тест и ранжирование выполняются тем же кодом, что и в приложении.

Команда: `python -m scripts.validate --out ../docs/validation_report.md` из каталога `backend`. Результат воспроизводится при том же значении seed. Полный отчёт приведён в приложении Б.

6.2. Главные результаты

Тестирование, 2000 кандидатов:

Показатель	Значение
Длина теста	от 12 до 18 заданий, в среднем 14,9
Верное решение «прошёл / не прошёл»	81%; для кандидатов не у самой границы грейда — 94%
Прошли, хотя уровень ниже заявленного	23%
Два прохождения дали одно решение	76%
Категория после всех попыток совпала с истинным грейдом	75% (выше истинного — 15%, ниже — 10%)
Корреляция балла с истинным уровнем	0,79
Надёжность балла: корреляция двух прохождений	0,65
Прошли, отвечая наугад	стажёр — 7%, junior, middle и senior — 0%
Слабый кандидат, заучивший 10 чужих тестов, прошёл	32% (без утечки — 6%)
То же при 200 утёкших тестах: до и после мониторинга дрейфа	83% и 46%
Задания, напрасно снятые мониторингом на честных ответах	0 из 84

Подбор, 164 запроса; релевантность — по скрытой истине (та же специализация и грейд, не менее 2/3 стека):

Способ ранжирования	Подходят в топ-10	Слабее запроса в топ-10
Случайный порядок в специализации	13%	39%
По резюме: заявленные грейд и стек	62%	18%
Платформа	60%	8%

6.3. Как это читать

- **Платформа устойчива к нечестным резюме.** Доля подходящих в топе у неё почти не зависит от того, сколько кандидатов завышают грейд (59–63%), а у ранжирования по резюме падает с 68% до 40%. При честных резюме платформа им уступает (61% против 68% при 20% завышающих), при 35% — почти равна, преимущество появляется, когда завышающих около половины.
- **Главный выигрыш — отсев слабых.** Кандидатов ниже запрошенного уровня в топе в 1,5–4 раза меньше. Именно на них работодатель тратит время технических специалистов.
- **Тест не безошибочен.** Один тест принимает верное решение по заявленному грейду в 81% случаев, а из кандидатов с уровнем ниже заявленного проходят 23%; категория после всех попыток верна в 75% случаев, ошибки почти всегда в пределах соседнего грейда. Это предел текущего размера банка.
- **Балл теста — грубая оценка.** Тест настроен на решение по грейду, а не на точный балл: надёжность балла 0,65. Поэтому в ранжировании балл только упорядочивает равных по соответствию.
- **Утечка снижена, но не исключена.** При 50 утёкших тестах слабый кандидат проходит в 56% случаев; мониторинг дрейфа срабатывает после накопления ответов и при массовой утечке снижает долю проходящих с 83% до 46%, но не до исходных 6%.

Ограничения процедуры: данные синтетические, а разбор вакансий проверялся на текстах, написанных вместе с правилами. Симуляция проверяет алгоритмы и их устойчивость к ошибкам калибровки, но не качество формулировок заданий — оно проверяется только на ответах реальных людей.

7. Интеграция с ФСП

7.1. Какие сведения используются

API и структура данных реестра не предоставляются, поэтому зафиксирован минимальный контракт (`backend/app/modules/fsp/registry.py`). По идентификатору участника реестр возвращает список достижений:

Поле	Смысл
event	Название соревнования
discipline	Дисциплина спортивного программирования
level	Уровень: региональный, федеральный округ, всероссийский
place	Место; пусто — участие без призового места
date	Дата соревнования
team	Командный результат или личный

Отдельным запросом реестр возвращает спортивный разряд участника. Разряд показывается в профиле и в карточке кандидата, но в оценке не участвует: он следует из тех же результатов, которые уже учтены.

Этого достаточно для обеих задач: показать работодателю проверяемое подтверждение практических навыков и учесть его в ранжировании. Копия достижений хранится в профиле и обновляется по кнопке «Обновить из реестра».

7.2. Случай без истории ФСП

Различаются три состояния, и ни одно не штрафуется:

Состояние	В кабинете кандидата	В выдаче работодателю
ФСП ID не привязан	Предложение привязать и пояснение, что это необязательно	Блок ФСП не показывается
ID привязан, достижений нет	«В реестре пока нет ваших результатов. Это не влияет на профиль»	Блок ФСП не показывается
Есть достижения	Список достижений	Метка, список, надбавка в оценке и строка в объяснении

Один ФСП ID привязывается только к одному профилю. Ввести чужой идентификатор нельзя: поля для него нет, привязка возможна только через вход в ФСП ID.

7.3. Вход через ФСП ID

ФСП ID — единый аккаунт участника на ресурсах Федерации (сервер Keycloak, протокол OpenID Connect). Доступа к нему и к тестовому стенду на хакатоне нет, поэтому провайдер заменён моком, а платформа проходит тот же путь, что и с настоящим сервером, — Authorization Code Flow:

```
1. Кандидат нажимает «Войти через ФСП ID» на экране входа либо «Привязать через ФСП ID» в настройках.
2. Бэкенд выдаёт адрес страницы входа провайдера с параметрами client_id, redirect_uri, scope и state.
   state подписан платформой, живёт 10 минут и различает вход и привязку к открытому профилю.
3. Кандидат входит на странице ФСП ID (почта и пароль ФСП) — пароль ФСП платформа не видит и не хранит.
4. Провайдер возвращает кандидата на платформу с одноразовым кодом и тем же state.
5. Бэкенд проверяет state, обменивает код на сведения об участнике: идентификатор (claim sub) и почту.
6. Бэкенд запрашивает в реестре достижения и разряд участника и сохраняет копию в профиле.
7. Вход: открывается сессия платформы. Привязка: ФСП ID записывается в профиль текущего кандидата.
```

Что происходит с учётной записью:

Случай	Результат
Первый вход через ФСП ID	Создаётся учётная запись кандидата с почтой из ФСП ID. Почту подтверждает провайдер, пароля на платформе у записи нет
Повторный вход	Та же учётная запись — по идентификатору участника
Почта уже зарегистрирована на платформе по паролю	Вход отклоняется: записи не объединяются по совпадению почты. Нужно войти по паролю и привязать ФСП ID в настройках

Случай	Результат
ФСП ID уже привязан к другому профилю	Привязка отклоняется
Привязку завершает не тот, кто её начал	Отклоняется: state содержит пользователя и сверяется с сессией

Регистрация по почте и паролю остаётся вторым способом входа: работодатели регистрируются только так, а кандидату без истории в ФСП аккаунт Федерации не нужен. Роли и данные хранятся на стороне платформы, провайдер подтверждает только личность.

Демонстрационный ФСП ID. Страница входа мока — отдельный экран `/fsp-id` с подписью «Демонстрационный стенд». Войти можно с любой почтой и общим паролем `fsp-demo`: идентификатор участника выводится из почты, поэтому одна почта всегда даёт одного участника с одним набором достижений (у части участников достижений нет). Готовые участники с известной историей перечислены в разделе 11.3.

Что нужно для подключения настоящего ФСП ID. Реализовать два метода контракта `FspIdProvider` (`backend/app/modules/fsp/idp.py`): `authorize_url` — адрес `/protocol/openid-connect/auth` сервера Keycloak с PKCE, `exchange` — обмен кода на токены в `/protocol/openid-connect/token` и проверка подписи ID-токена; и два метода `FspRegistry` поверх API реестра с токеном доступа. Маршруты платформы, state, правила создания и привязки учётных записей, интерфейс — не меняются. Обновление достижений тогда выполняется по refresh-токену либо по уведомлению от реестра; сейчас — по кнопке «Обновить из реестра».

8. Безопасность и персональные данные

- **Пароли** хранятся в виде хешей argon2. Токен доступа — JWT в cookie с флагами httpOnly и SameSite=Lax (и Secure при работе по HTTPS), из скриптов страницы недоступен. Токен привязан к сессии на сервере (таблица `auth_sessions`): выход удаляет сессию, и скопированный до выхода токен больше не принимается, хотя его подпись и срок ещё действительны. Выход закрывает одну сессию, остальные устройства остаются в системе.
- **Регистрация** — по почте с подтверждением адреса; без подтверждения вход закрыт. Второй способ для кандидата — вход через ФСП ID (раздел 7.3): пароль ФСП платформа не получает, одноразовый код живёт две минуты, а совпадение почты не даёт доступа к уже существующей учётной записи.
- **Роли** разделены на уровне API: каждый маршрут кабинета проверяет роль, чужие записи отвечают 404.
- **Согласие (152-ФЗ).** Согласие на обработку обязательно при регистрации, дата сохраняется. Согласие на публикацию профиля — отдельное и отзывается в один клик. Отзыв согласия на обработку удаляет учётную запись и все связанные данные.
- **Минимизация данных.** Файл резюме не сохраняется: из него берутся только поля, которые кандидат подтвердил. Верные ответы на задания клиенту не передаются.
- **Приватность на всех путях.** Правила видимости собраны в одном модуле (`candidates/privacy.py`) и применяются везде, где сведения о кандидате отдаются работодателю: в банке и подборке, в отклике на вакансию и в ответах на задания. Скрытый стаж не показывается и в отклике. Из текста «О себе» и из ответов на задания вырезаются почта, телефон, ники и ссылки: до согласия кандидата работодатель контактов не видит. Правила ловят типовые записи контактов, а не любой способ передать их словами.
- **Секреты** задаются переменными окружения. Значения по умолчанию у `SECRET_KEY` нет: приложение не стартует, если секрет не задан, короче 32 символов или совпадает с примером из старых версий.
- **Одновременные запросы.** Ограничения, на которых держится сценарий, обеспечивает база, а не проверка в приложении: один незавершённый тест на кандидата и одно действующее приглашение на пару «работодатель — кандидат» — частичные уникальные индексы; из двух одновременных ответов на одно задание сохраняется один (версия строки, оптимистическая блокировка); повторный отклик или ответ на задание получает 409.
- **Проверка ввода.** Пробелы по краям строк отбрасываются до проверки длины; суммы, идентификаторы, длина тегов и стаж ограничены сверху; номер варианта ответа сверяется с заданием до изменения попытки. Сообщения об ошибках ввода — на русском и без повторения присланных значений.
- **Ограничение частоты.** Письмо подтверждения — не чаще раза в минуту и пяти раз в час на адрес, 20 запросов в час с одного источника; вход — не больше 10 неудачных попыток на адрес за 15 минут. Счётчики хранятся в памяти процесса: для нескольких экземпляров приложения их нужно вынести в общее хранилище.
- **Сбой почты.** Если письмо подтверждения отправить не удалось, учётная запись не создаётся (ответ 503), и регистрацию можно повторить; письмо можно запросить заново с экранов регистрации и входа.

Не реализовано в MVP: общий лимит частоты для остальных маршрутов, журнал доступа к персональным данным, восстановление пароля.

9. Концепции за рамками MVP

9.1. Защита от фиктивных вакансий и недобросовестных работодателей

Обратная механика усиливает обычный риск: работодатель получает доступ к базе проверенных кандидатов. Предлагаемая защита — три уровня.

Проверка при входе. Подтверждение компании по ИНН через ЕГРЮЛ и корпоративной почте на домене компании; для ИП и новых компаний — ручная проверка. Статус «проверена» показывается кандидату в приглашении.

Ограничения по поведению. Новому работодателю — лимит приглашений в сутки, который растёт вместе с репутацией. Репутация считается по уже имеющимся в системе данным:

- доля принятых и отклонённых приглашений, доля оставшихся без ответа после раскрытия контактов;
- жалобы кандидатов (кнопка в приглашении: «не соответствует описанию», «спам», «запрос оплаты»);
- массовые однотипные приглашения кандидатам разных категорий;
- вилка, резко выше рыночной для категории, — частый признак приманки.

Реакция. При срабатывании признаков приглашения уходят на ручную модерацию, а контакты принявших кандидатов временно не раскрываются. Кандидат видит в приглашении возраст учётной записи компании и её статус проверки.

Данные для первого шага уже собираются: статусы приглашений и откликов, время ответа, зарплатные вилки.

9.2. Интеграция с ATS работодателей

Цель — чтобы рекрутёр работал в привычной системе, а платформа была источником проверенных кандидатов.

- **Исходящие события (webhooks).** Платформа сообщает ATS о событиях «приглашение принято», «новый отклик», «новый кандидат в подборке». В событии — обезличенная карточка, а после раскрытия контактов — и они. Подпись события общим секретом, повтор при недоставке.
- **Входящий API.** ATS создаёт потребность или вакансию и получает подборку теми же методами, что и интерфейс; авторизация — ключом компании (OAuth 2.0 Client Credentials).
- **Соответствие статусов.** Этапы воронки ATS отображаются на статусы платформы; итог (оффер, отказ) возвращается обратно и становится обратной связью для кандидата и сигналом для репутации работодателя.
- **Коннекторы.** Адаптеры для распространённых систем (Huntflow, Talantix, Potok) и универсальный вариант — webhooks и выгрузка в CSV.

API уже описан спецификацией OpenAPI, поэтому входящая интеграция не требует отдельного контракта.

10. Используемые библиотеки и компоненты

Весь код открыт, не скомпилирован и не обфусцирован. Все зависимости — с открытыми лицензиями.

Backend (Python 3.13):

Компонент	Версия	Лицензия	Назначение
FastAPI	0.142.2	MIT	Веб-фреймворк, генерация OpenAPI
Uvicorn	0.54.0	BSD-3-Clause	ASGI-сервер
SQLAlchemy	2.1.3	MIT	ORM
psycopg	3.3.6	LGPL-3.0	Драйвер PostgreSQL
Pydantic, pydantic-settings	2.13.5, 2.15.0	MIT	Валидация данных и настройки
PyJWT	2.15.1	MIT	Токены доступа
argon2-cffi	25.1.0	MIT	Хеширование паролей
python-multipart	0.0.32	Apache-2.0	Приём файлов
pypdf	6.19.0	BSD-3-Clause	Извлечение текста из PDF-резюме
fpdf2	2.8.9	LGPL-3.0	Формирование PDF-профиля и этой документации
Markdown	3.11	BSD-3-Clause	Сборка документации
pytest, httpx	9.1.1, 0.28.1	MIT, BSD-3-Clause	Тесты

Frontend (Node.js 20):

Компонент	Версия	Лицензия	Назначение
React, React DOM	19.3.0	MIT	Интерфейс
TypeScript	5.9.3	Apache-2.0	Типизация
Mantine (core, form, hooks, notifications)	8.3.18	MIT	Компоненты интерфейса
TanStack Query	5.104.1	MIT	Работа с данными сервера
React Router	7.18.4	MIT	Маршрутизация
Vite, @vitejs/plugin-react	6.4.3, 4.7.0	MIT	Сборка
@fontsource/jetbrains-mono	5.3.0	OFL-1.1	Фирменный шрифт JetBrains Mono

Инфраструктура:

Компонент	Версия	Лицензия	Назначение
PostgreSQL	17	PostgreSQL License	СУБД
nginx	1.27	BSD-2-Clause	Раздача интерфейса, прокси к API
Mailpit	latest	MIT	Почтовая ловушка для проверки писем
Шрифты DejaVu	—	Bitstream Vera / свободная	Кириллица в PDF

11. Сборка, развёртывание и запуск

11.1. Запуск в Docker

Нужны Docker и Docker Compose. Оценка потребления памяти — около 1 ГБ на все контейнеры.

```
1. Скопировать файл настроек:        cp .env.example .env
2. Задать в .env значение SECRET_KEY (не короче 32 символов):
     python -c "import secrets; print(secrets.token_urlsafe(48))"
3. Собрать и запустить:               docker compose up --build -d
4. Наполнить демонстрационными данными (необязательно):
     docker compose exec backend python -m scripts.seed --reset
```

После запуска:

Адрес	Что там
http://localhost:8080	Приложение
http://localhost:8080/api/docs	Описание API (Swagger)
http://localhost:8025	Письма с подтверждением регистрации (Mailpit)

Для размещения на сервере в `.env` задаются `PUBLIC_URL` (внешний адрес — он подставляется в ссылки из писем), `COOKIE_SECURE=true` при работе по HTTPS и параметры настоящего SMTP-сервера (`SMTP_HOST`, `SMTP_PORT`). HTTPS обеспечивает внешний обратный прокси перед портом 8080.

Сборка в Docker на машине разработчика не проверялась (Docker на ней отсутствует); проверены локальный запуск и автоматические тесты.

11.2. Локальный запуск без Docker

Нужны Python 3.13 и Node.js 20. Используется SQLite, письма выводятся в журнал бэкенда.

```
# Бэкенд (из корня репозитория; на Linux и macOS — .venv/bin/python)
python -m venv backend/.venv
backend/.venv/Scripts/python -m pip install -r backend/requirements.txt
backend/.venv/Scripts/python -m uvicorn app.main:app --app-dir backend --port 8100

# Интерфейс (во втором терминале)
npm --prefix frontend install
npm --prefix frontend run dev
```

Приложение: http://localhost:5183, описание API: http://localhost:8100/api/docs.

Демонстрационные данные (команда удаляет все данные в базе; на Linux и macOS переменная задаётся так же):

```
PYTHONPATH=backend backend/.venv/Scripts/python -m scripts.seed --reset
```

11.3. Демонстрационные учётные записи

Пароль у всех — `demo12345`.

Почта	Роль и состояние
demo-newcomer@example.com	Новый кандидат без теста — чтобы пройти путь с начала
demo-newcomer2@example.com … demo-newcomer5@example.com	Такие же чистые записи, по одной на проверяющего
demo-candidate@example.com	Кандидат с категорией «Бэкенд · Middle» и входящими приглашениями
demo-employer@example.com	Работодатель с потребностями, вакансиями, откликами и заданием

В базе также около 300 синтетических кандидатов, категории которых получены прогоном настоящего теста; несколько из них тест не прошли и видны работодателю со статусом «грейд не подтверждён».

Вход через демонстрационный ФСП ID — кнопка «Войти через ФСП ID», пароль у всех `fsp-demo`:

Почта	Участник
champion@fsp.example	Шесть соревнований, пять призовых мест, разряд
participant@fsp.example	Два участия без призовых мест
newcomer@fsp.example	Аккаунт без истории соревнований
любая другая почта	Участник со случайной, но постоянной для этой почты историей

Как эксперту пройти тест самому. На стенде проверки задаётся `DEMO_MODE=true`. Тогда можно зарегистрироваться с любой почтой без письма с подтверждением либо войти через ФСП ID, пройти тест, а затем кнопкой «Сбросить попытки и пройти заново» на экране теста обнулить результат и пройти его ещё раз — на тот же или другой грейд. Тест — от 12 до 18 заданий, до 30 минут. Без демо-режима письмо с подтверждением приходит в Mailpit, а повторный тест ограничен сроками из раздела 4.3.

11.4. Проверки

```
cd backend
.venv/Scripts/python -m pytest                                    # автоматические тесты
.venv/Scripts/python -m scripts.validate --out ../docs/validation_report.md   # отчёт валидации, ~2 минуты
.venv/Scripts/python -m scripts.build_docs                        # сборка этой документации в PDF
cd ../frontend && npm run build                                   # проверка типов и сборка интерфейса
```

11.5. Настройки бэкенда

Переменная	По умолчанию	Смысл
DATABASE_URL	sqlite:///./dev.db	Строка подключения к базе
SECRET_KEY	нет, обязателен	Секрет подписи токенов, не короче 32 символов; без него приложение не стартует
FRONTEND_URL	http://localhost:5183	Адрес интерфейса для ссылок в письмах
COOKIE_SECURE	false	Отдавать cookie только по HTTPS
SMTP_HOST, SMTP_PORT	не задан, 1025	Почтовый сервер; без него письма пишутся в журнал
GRADE_CHANGE_COOLDOWN_DAYS	90	Как часто можно менять подтверждённую категорию
RETAKE_COOLDOWN_DAYS	14	Через сколько можно пересдать проваленный грейд
TEST_DURATION_MIN	30	Время на тест

Переменная	По умолчанию	Смысл
DEMO_MODE	false	Стенд проверки: регистрация без подтверждения почты, кандидат может сбросить свои попытки теста
PDF_FONT_DIR	не задан	Каталог со шрифтами DejaVu, если их нет в системе

12. Известные ограничения

- Языковые модели не используются; разбор резюме и потребности выполнен правилами.
- Банк заданий невелик и не откалиброван на реальных ответах; от его размера зависят и точность теста, и устойчивость к утечке.
- ФСП ID и реестр ФСП заменены моками: вход проходит по схеме OpenID Connect, но провайдер демонстрационный и принимает любую почту с общим паролем; достижения участников выдуманы.
- У учётной записи, созданной через ФСП ID, нет пароля платформы: войти в неё можно только через ФСП ID.
- Изменения схемы базы не сопровождаются миграциями: схема создаётся при старте.
- Мобильная вёрстка адаптивна, но на устройствах отдельно не тестировалась.
- Защита от недобросовестных работодателей и интеграция с ATS описаны как концепция.

Приложение А. Описание API

Все методы начинаются с `/api`, принимают и возвращают JSON (кроме загрузки резюме и выгрузки PDF). Авторизация — cookie `access_token`, которую выставляет метод входа. Общие коды ошибок: 401 — не выполнен вход, 403 — раздел другой роли, 422 — ошибка в данных запроса (текст — в поле `detail`). Полные схемы запросов и ответов — в интерактивном описании `/api/docs` и в `/api/openapi.json`.

Авторизация

Метод и путь	Назначение	Свои коды ошибок
POST /api/auth/register	Регистрация по электронной почте	409 — Почта уже зарегистрирована; 503 — Не удалось отправить письмо; учётная запись не создана
POST /api/auth/verify	Подтверждение адреса почты по токену из письма	400 — Ссылка недействительна или устарела
POST /api/auth/resend-verification	Повторная отправка письма (не чаще раза в минуту на адрес)	429 — Слишком много запросов
POST /api/auth/login	Вход; токен доступа выдаётся в httpOnly-cookie	401 — Неверная почта или пароль; 403 — Почта не подтверждена; 429 — Слишком много запросов
POST /api/auth/logout	Выход: сессия закрывается на сервере	—
GET /api/auth/me	Текущий пользователь	—
DELETE /api/auth/me	Отозвать согласие: удалить учётную запись и все связанные данные	—

Справочники

Метод и путь	Назначение	Свои коды ошибок
GET /api/reference	Специализации, грейды, форматы работы и словарь технологий	—

Кабинет кандидата

Метод и путь	Назначение	Свои коды ошибок
GET /api/candidate/profile	Свой профиль	—
PUT /api/candidate/profile	Сохранить профиль	—
POST /api/candidate/profile/resume	Распознать резюме из PDF: возвращает черновик полей профиля, ничего не сохраняя	413 — Файл слишком большой; 422 — Файл не PDF, повреждён или не содержит текста
GET /api/candidate/profile/pdf	Стандартизированный PDF-профиль кандидата	503 — На сервере нет шрифта с кириллицей

Кабинет работодателя

Метод и путь	Назначение	Свои коды ошибок
GET /api/employer/company	Профиль компании	—
PUT /api/employer/company	Сохранить профиль компании	—

Тестирование и категория

Метод и путь	Назначение	Свои коды ошибок
GET /api/candidate/assessment	Категория, текущий тест и история попыток	—
POST /api/candidate/assessment	Опрос и начало теста на выбранный грейд	409 — Тест уже идёт либо грейд пока недоступен
POST /api/candidate/assessment/{attempt_id}/answer	Ответить на текущее задание: выдаётся следующее либо тест завершается с присвоением категории	404 — Попытка не найдена; 409 — Тест завершён, время истекло либо ответ дан не на текущее задание; 422 — Такого варианта ответа в задании нет
POST /api/candidate/assessment/reset	Демо-режим: удалить свои попытки и категорию, чтобы пройти тест заново	404 — Стенд работает не в демо-режиме

Связь с ФСП ID

Метод и путь	Назначение	Свои коды ошибок
GET /api/auth/fsp/start	Начать вход через ФСП ID	—
GET /api/candidate/fsp/start	Начать привязку ФСП ID к профилю: владение подтверждается входом в ФСП ID	—

Метод и путь	Назначение	Свои коды ошибок
POST /api/auth/fsp/callback	Завершить вход или привязку: обмен кода ФСП ID на сведения об участнике	400 — Код или state недействительны либо устарели; 401 — Привязку завершает не тот пользователь, который её начал; 409 — ФСП ID привязан к другому профилю либо почта уже зарегистрирована
POST /api/candidate/fsp/refresh	Обновить достижения из реестра ФСП	409 — ФСП ID не привязан
DELETE /api/candidate/fsp	Отвязать ФСП ID	409 — Учётная запись создана через ФСП ID: другого способа входа нет

Демонстрационный ФСП ID (мок провайдера)

Метод и путь	Назначение	Свои коды ошибок
POST /api/fsp-id/login	Страница входа демонстрационного ФСП ID: проверка пароля и выдача кода	401 — Неверный пароль
GET /api/fsp-id/accounts	Подсказка для страницы входа: демо-участники	—

Подбор кандидатов

Метод и путь	Назначение	Свои коды ошибок
GET /api/employer/candidates	Банк кандидатов: вся база с фильтрами, сортировка по силе подтверждённого профиля; в конце — кандидаты с неподтверждённым грейдом	—
POST /api/employer/needs/parse	Разобрать текст потребности: специализация, грейд, стек, формат, зарплата	—
GET /api/employer/needs	Сохранённые потребности	—
POST /api/employer/needs	Сохранить потребность	—
PUT /api/employer/needs/{need_id}	Изменить потребность	404 — Потребность не найдена
DELETE /api/employer/needs/{need_id}	Удалить потребность	404 — Потребность не найдена
GET /api/employer/needs/{need_id}/matches	Подборка под потребность: рекомендованные категории и ранжированные кандидаты с объяснением	404 — Потребность не найдена

Приглашения

Метод и путь	Назначение	Свои коды ошибок
GET /api/employer/invitations	Отправленные приглашения и их статусы	—

Метод и путь	Назначение	Свои коды ошибок
POST /api/employer/invitations	Пригласить кандидата: предложение, зарплатная вилка и способ связи	404 — Кандидат не найден в банке; 409 — Не заполнено название компании либо кандидат уже приглашён
GET /api/candidate/invitations	Входящие приглашения; новые после выдачи помечаются просмотренными	—
POST /api/candidate/invitations/{invitation_id}/accept	Принять приглашение: контакты кандидата открываются работодателю	404 — Приглашение не найдено; 409 — На приглашение уже дан ответ
POST /api/candidate/invitations/{invitation_id}/decline	Отклонить приглашение	404 — Приглашение не найдено; 409 — На приглашение уже дан ответ

Вакансии и отклики

Метод и путь	Назначение	Свои коды ошибок
GET /api/employer/vacancies	Вакансии компании	—
POST /api/employer/vacancies	Опубликовать вакансию	409 — Не заполнено название компании
PUT /api/employer/vacancies/{vacancy_id}	Изменить вакансию, закрыть или открыть заново	404 — Вакансия не найдена
GET /api/employer/vacancies/{vacancy_id}/applications	Отклики на вакансию; новые после выдачи помечаются просмотренными	404 — Вакансия не найдена
POST /api/employer/applications/{application_id}/accept	Принять отклик	404 — Отклик не найден; 409 — На отклик уже дан ответ
POST /api/employer/applications/{application_id}/decline	Отклонить отклик	404 — Отклик не найден; 409 — На отклик уже дан ответ
GET /api/candidate/vacancies	Открытые вакансии; подходящие категории кандидата идут первыми	—
POST /api/candidate/vacancies/{vacancy_id}/apply	Откликнуться на вакансию: контакты кандидата открываются этому работодателю	404 — Вакансия не найдена или закрыта; 409 — Категория ещё не подтверждена тестом либо отклик уже отправлен
GET /api/candidate/applications	Свои отклики и их статусы	—

Регулярные задания

Метод и путь	Назначение	Свои коды ошибок
GET /api/employer/challenges	Задания компании	—
POST /api/employer/challenges	Опубликовать задание для кандидатов специализации	409 — Не заполнено название компании
PUT /api/employer/challenges/{challenge_id}	Изменить или закрыть задание	404 — Задание не найдено

Метод и путь	Назначение	Свои коды ошибок
GET /api/employer/challenges/{challenge_id}/answers	Ответы кандидатов на задание (обезличенные)	404 — Задание не найдено
PUT /api/employer/answers/{answer_id}/rating	Оценить ответ от 1 до 5	404 — Ответ не найден
GET /api/candidate/challenges	Задания по специализации кандидата, на которые он ещё не отвечал	—
POST /api/candidate/challenges/{challenge_id}/answer	Ответить на задание: решение или подход к нему	404 — Задание не найдено или закрыто; 409 — Ответ уже отправлен
GET /api/candidate/answers	Свои ответы и оценки работодателей	—

Служебное

Метод и путь	Назначение	Свои коды ошибок
GET /api/health	Проверка доступности	—

Приложение Б. Полный отчёт валидации

Отчёт собран автоматически командой `python -m scripts.validate --n 2000 --seed 1` из каталога `backend` и воспроизводится при том же значении seed. Методика — в описаниях модулей `scripts/synth.py`, `scripts/validate_testing.py` и `scripts/validate_matching.py`.

Данные синтетические: у каждого кандидата есть скрытые истинный уровень и реальные навыки, а платформа видит только заявленные им грейд и стек и его ответы на задания. Все показатели — сравнение выводов платформы со скрытой истиной. Ответы на задания генерируются по модели IRT, поэтому отчёт проверяет алгоритмы теста и подбора и их устойчивость к ошибкам калибровки и нечестным резюме, но не заменяет калибровку заданий на реальных ответах.

Тестирование

Симуляция 2000 кандидатов с известным уровнем. «Идеальная калибровка» — истинная сложность заданий равна назначенной; «реалистичная» — отличается от неё (сдвиг сложности со стандартным отклонением 0,4 грейда, разброс различающей способности ±25%). Остальные показатели — для реалистичной.

Тест переменной длины: от 12 до 18 заданий, в среднем 14.9. На наименьшей длине заканчивают 43% тестов (результат очевиден), до наибольшей доходят 41% (кандидат у самой границы грейда).

Точность оценки (один тест на заявленный грейд)

Показатель	Идеальная калибровка	Реалистичная
Корреляция оценки с истинным уровнем	0.80	0.79
Ошибка оценки (RMSE), в грейдах	0.69	0.72
Смещение оценки, в грейдах	+0.03	+0.11
Грейд по результату совпал с истинным	53%	50%
…или отличается не более чем на один	95%	94%
Верное решение «прошёл / не прошёл»	80%	81%
…для кандидатов не ближе 0,5 грейда к границе	93%	94%
Прошли, хотя уровень ниже заявленного	20%	23%
Не прошли, хотя уровень соответствует	20%	17%

Повторяемость (два независимых прохождения)

- Корреляция оценок двух прохождений (надёжность): 0.65
- Одинаковое решение «прошёл / не прошёл»: 76%
- Одинаковый грейд по результату: 44%
- Среднее расхождение балла: 11 из 100

Итог категоризации (заявленный грейд; при провале — грейд ниже, при уверенном результате — выше)

- Категория совпала с истинным грейдом: 75%
- Категория выше истинного грейда: 15% (на два грейда и больше — 0%)
- Категория ниже истинного грейда или не присвоена: 10%
- Среднее число попыток на кандидата: 1.80

Разделение слабых и сильных

Средний балл теста по истинному грейду: intern — 35, junior — 47, middle — 55, senior — 70.

Вероятность того, что кандидат старшего грейда наберёт больше кандидата соседнего младшего (AUC): intern → junior — 0.77, junior → middle — 0.71, middle → senior — 0.81.

Угадывание

Доля прошедших среди отвечающих наугад: intern — 7%, junior — 0%, middle — 0%, senior — 0%.

Устойчивость к распространению заданий

Задания	Разных вариантов из 300 выдач (мин. / медиана)	Два кандидата видят задание одинаково
Общие (параметризованные)	71 / 280	0%
Профильные	4 / 4	18%

Слабый кандидат (на грейд ниже границы) заучил ответы из K чужих тестов на тот же грейд:

Утёкших тестов	Узнано заданий в своём тесте	Прошли тест
0	0%	6%
10	30%	32%
50	43%	56%
200	57%	82%

Для сравнения: при едином наборе заданий одного утёкшего теста достаточно, чтобы пройти наверняка.

Мониторинг дрейфа

Задание снимается с выдачи, если его решают заметно чаще, чем следует из уровня кандидатов по общим (генерируемым) заданиям: не меньше 30 ответов, превышение больше 15 п.п. и больше 2.5 стандартного отклонения.

- Без утечки: из 84 профильных заданий с достаточным числом ответов снято напрасно — 0
- С утечкой 200 тестов (271 честных попыток и 400 попыток списывающих по одной специализации): снято 7 из 12 профильных заданий
- Списывающие проходят тест: до мониторинга — 83%, после — 46% (узнают 33% заданий)

Показы заданий

- Общее задание встречается в 39% тестов (медиана), максимум — 39%
- Профильное задание встречается в 42% тестов (медиана), максимум — 46%

Подбор

Разбор потребности из текста

14 описаний вакансий с ручной разметкой (`backend/scripts/vacancies.py`):

- специализация определена верно: 100%
- грейд определён верно: 100%
- стек: точность 98%, полнота 100%
- формат работы: 100%
- зарплатная вилка (включая её отсутствие): 100%

Тексты писались вместе с правилами разбора, поэтому это проверка на согласованность, а не оценка на независимых данных: на произвольных текстах точность будет ниже.

Доля релевантных кандидатов в топе выдачи

Синтетический банк: 2000 кандидатов, из них 1929 получили категорию по тесту. Запросов: 164 (разобранные тексты вакансий и случайные сочетания специализации, грейда и трёх технологий). Релевантность считается по скрытой истине — истинному грейду и реальным навыкам: «подходит» — та же специализация и грейд и не менее 2/3 требуемого стека; «частично» — смежный грейд либо от 1/3 стека. Подходящих в среднем 13% специализации. «Слабее запроса» — истинный грейд кандидата ниже запрошенного.

Способ ранжирования	Подходят в топ-5	Подходят в топ-10	Хотя бы частично в топ-10	Слабее запроса в топ-10	NDCG@10
Случайный порядок в специализации	14%	13%	44%	39%	0.24
По резюме: заявленные грейд и стек	65%	62%	92%	18%	0.76
Платформа: категория по тесту и ранжирование	63%	60%	94%	8%	0.75

Чувствительность к честности резюме

Тот же расчёт при разной доле кандидатов, завышающих грейд в самооценке (в основной таблице — 35%):

Завышают грейд	Подходят в топ-10: резюме	Подходят в топ-10: платформа	Слабее запроса в топ-10: резюме	Слабее запроса в топ-10: платформа
20%	68%	61%	12%	8%
35%	62%	60%	18%	8%
50%	52%	63%	29%	9%
65%	40%	59%	37%	9%